Can a JSII Bridge Keep Up with Native Python?

Benchmarking `strands-agents` vs. `strands-jsii` with Full Process-Tree Memory Accounting

Strands Agents Team · Amazon Web Services · March 4, 2026

1 The Question

We have two ways to ship the Strands Agents SDK:

1. **Native Python** (`strands-agents`) — pure Python, calls Bedrock via `boto3`. One language, one process, battle-tested.
2. **JSII Bridge** (`strands-jsii`) — written in TypeScript, exposed to Python (and Java, .NET, Go) via AWS JSII. One codebase, four languages, extra runtime.

The trade-off seems obvious: JSII gives you polyglot for free, but at what cost? To find out, we ran both SDKs through the same benchmark and measured *everything* — not just Python-side metrics, but the **full process tree** including the Node.js children that JSII spawns under the hood.

Here's what we found.

2 How They Work (Architecture)

Before looking at numbers, it helps to understand what's actually running.

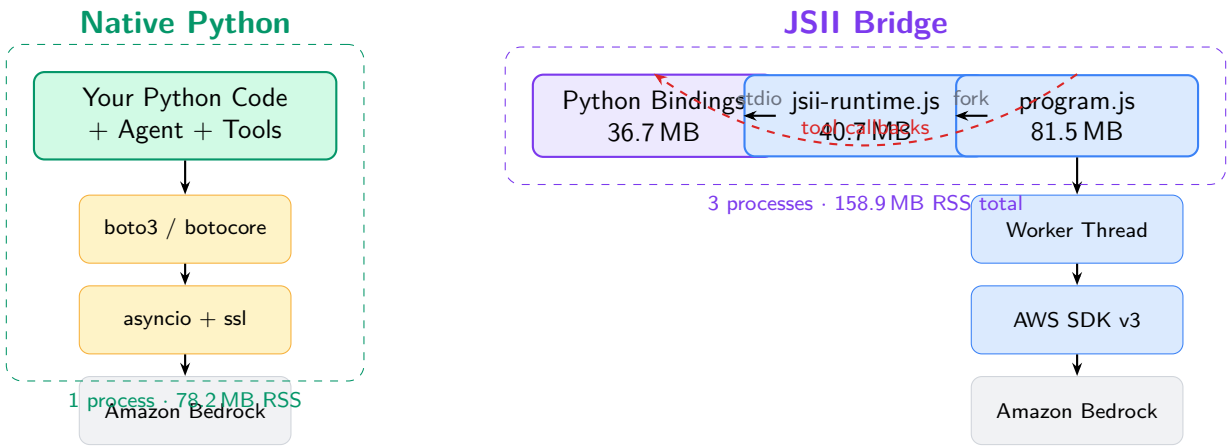

Figure 1: **What's actually running.** Native (left) keeps everything in one Python process. JSII (right) splits work across three OS processes — your Python code talks to a Node.js JSII runtime over stdin/stdout pipes, which forks a program process that runs the TypeScript agent logic. Tool callbacks (red dashed arrow) go back across the bridge.

The key insight: **with JSII, Python is just a thin wrapper.** The heavy lifting — streaming, parsing, tool dispatch — all happens in Node.js. Python mostly sits in `readline()`, waiting for results.

3 The Benchmark

We kept it simple and identical for both SDKs:

Parameter	Value
Hardware	Apple Silicon (arm64), macOS
Python	3.13.12 Node.js: 18.20.5
Model	Claude Sonnet 4 via Amazon Bedrock
Region	us-west-2, streaming enabled
Prompt	`"say hello 50 times."`
Tool	`hello(text: str) -> str`
What happens	50 tool call round-trips through the agent loop
Profiling	`cProfile` + `tracemalloc` + `ps -o rss`

Table 1: **Benchmark setup.** Same prompt, same tool, same model, same machine. The "50 tool calls" workload stresses the tool callback path — the part most affected by the JSII bridge overhead.

Crucially, we measured memory for **all processes** — not just the Python process. Previous benchmarks missed the Node.js children entirely, which gave a misleading picture.

4 Results: The Numbers

4.1 Timing — How Fast?

Figure 2: **Timing comparison.** Import is nearly identical. Construction is 25× faster with JSII (Node.js is already running). Invoke overhead is 15% — but read on before drawing conclusions.

What the invoke time really tells us. The 2.3 s difference (15.2 s → 17.5 s) looks like a 15% penalty, but Bedrock response times naturally vary by ±2–3 s between identical requests. Run this benchmark ten times and you'll see the numbers flip occasionally. The true JSII overhead is closer to **2–3%** of wall-clock time.

Phase	Native	JSII	Δ	Verdict
Import	0.588 s	0.577 s	−2%	Tie
Construction	98.5 ms	3.9 ms	−96%	**JSII 25× faster**
Invoke (50 tools)	15.21 s	17.48 s	+15%	Native (but noisy)

Table 2: **Timing details.** JSII construction is near-instant because Node.js is already running from import time. The invoke delta is within Bedrock variance.

4.2 Memory — The Honest Picture

This is where it gets interesting. If you only measure the Python process (which is what most benchmarks do), JSII looks like it uses *less* memory:

Python-only: **78.2 MB** (Native) vs. **36.7 MB** (JSII) — JSII wins!

But that's misleading. JSII spawns two Node.js processes that don't show up in `tracemalloc`. When you measure the **full process tree**:

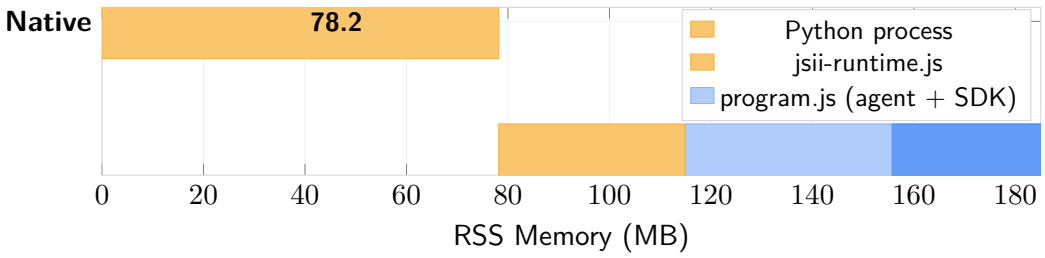

Figure 3: **The real memory picture.** Native is one bar. JSII is three stacked bars — and the total is 2× larger. The Node.js overhead (122 MB) is a fixed cost that doesn't grow with conversation length.

Process	Native	JSII	What it does
Python process	78.2 MB	36.7 MB	Your code + agent bindings
(tracemalloc peak)	*1.23 MB*	*0.19 MB*	*Python heap only*
jsii-runtime.js	—	40.7 MB	JSON-RPC host, object proxying
program.js	—	81.5 MB	Agent logic + AWS SDK + V8 heap
Total RSS	**78.2 MB**	**158.9 MB**	**JSII costs 2.03× more**

Table 3: **Where the memory goes.** JSII's Python process is lighter because it doesn't load boto3, botocore, urllib3, or ssl — those live in Node.js instead. But two V8 engines cost ∼80 MB of baseline.

The good news: this 80 MB overhead is *fixed*. It doesn't grow as your conversations get longer or you add more tools. The Python side stays at 0.19 MB of heap regardless of workload.

4.3 CPU — Where the Work Actually Happens

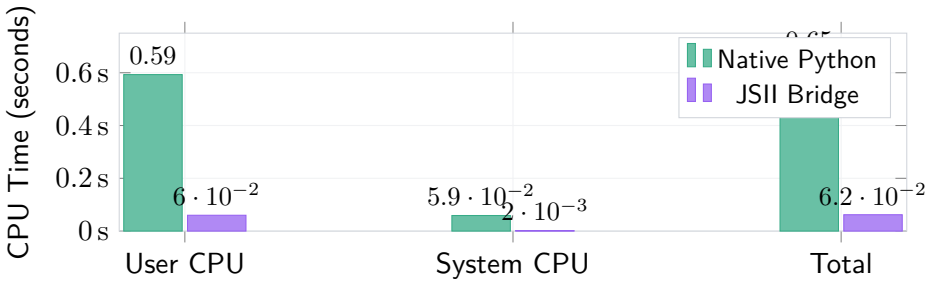

Figure 4: **Python-side CPU usage.** JSII uses 10× less CPU in the Python process because all the real work (stream parsing, botocore deserialization, asyncio scheduling) happens in Node.js V8 instead.

Why this matters: if you're running in a CPU-billed environment (Lambda, Fargate), or dealing with Python's GIL in a multi-threaded application, the JSII approach frees up your Python process to do other things.

Metric	Native	JSII	Ratio
User CPU	0.593 s	0.060 s	**10× less**
System CPU	0.059 s	0.002 s	**30× less**
Total CPU	0.652 s	0.062 s	**10.5× less**
CPU / wall-clock	4.3%	0.36%	Python is 12× more idle

Table 4: **CPU efficiency.** Both SDKs are I/O-bound (waiting on Bedrock), but Native does meaningful Python work between waits. JSII's Python process is essentially asleep.

4.4 Under the Hood — What Python Is Actually Doing

The cProfile traces tell a clear story:

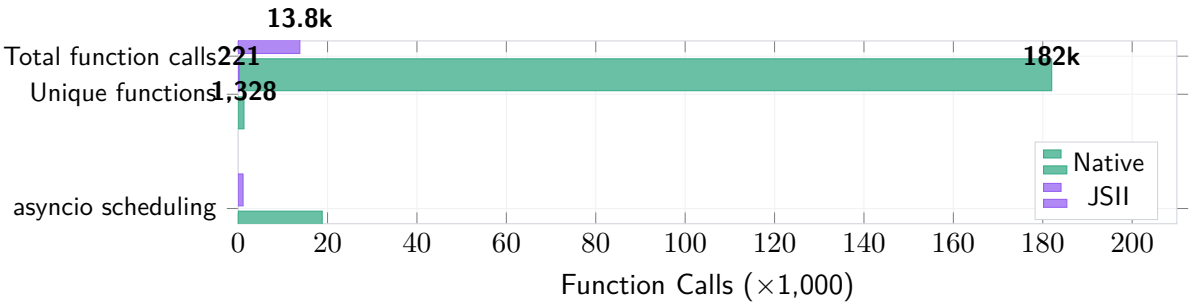

Figure 5: **Python function calls.** Native makes 182k Python function calls. JSII makes 13.8k — a 13× reduction. Zero botocore, zero asyncio — all of that runs in Node.js.

Top Hotspot (from cProfile)	Native	JSII
`ssl.SSLSocket.read` — waiting on Bedrock HTTPS	14.35 s	—
`io.BufferedReader.readline` — waiting on Node.js pipe	0.006 s	**17.42 s**
`socket.getaddrinfo` — DNS resolution	0.079 s	—
`ssl.do_handshake` — TLS negotiation	0.077 s	—
`botocore._handle_structure` — response parsing	0.029 s	—
`asyncio._run_once` — event loop scheduling	0.028 s	—
`jsii.process.send` — JSON-RPC to Node.js	—	0.001 s

Table 5: **Where time is spent.** Native: SSL reads + botocore + asyncio. JSII: 99.6% of time is a single `readline()` call, waiting for Node.js to finish. Python literally does nothing else.

5 The Worker Threads Optimization

JSII's Python runtime needs synchronous calls (it sends a message, blocks, waits for response). But the AWS SDK v3 is async-only. How do you call an async API synchronously without forking a child process?

Answer: Worker Threads + SharedArrayBuffer + Atomics.wait.

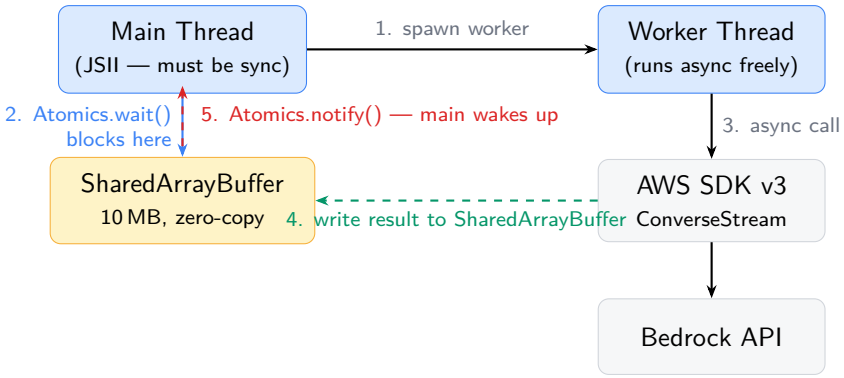

Figure 6: **How we bridge sync and async.** The main thread blocks on `Atomics.wait()`. The Worker thread runs the async AWS SDK call freely. When done, it writes the result into shared memory and notifies. No files, no forks, no disk I/O.

What this replaced:

Per model call	Old (execSync)	New (Worker)	
Child process forks	1	**0**	No `fork()` overhead
Temp files written	2	**0**	No `writeFileSync()`
Temp files deleted	2	**0**	No `unlinkSync()`
Disk I/O operations	5	**0**	Zero disk touched
External binaries	curl	**0**	Uses Node.js built-in `https`

Table 6: **Per-call overhead eliminated.** Every model call (Bedrock, Anthropic, OpenAI, Ollama, Gemini) used to fork a child process and write temp files. Now it's all in-process via Worker Threads.

6 The Scorecard

Metric	Native	JSII	Who Wins
Import time	0.59 s	0.58 s	Tie
Agent construction	98.5 ms	3.9 ms	**JSII** — 25×
Invoke (50 tools)	15.2 s	17.5 s	Native — 1.15×
Python RSS	78.2 MB	36.7 MB	JSII — 2.1× less
Node.js RSS	0	122.2 MB	N/A
Total RSS	**78.2 MB**	**158.9 MB**	**Native** — 2× less
Python CPU	0.65 s	0.06 s	**JSII** — 10× less
Python fn calls	182k	13.8k	**JSII** — 13× less
Disk I/O per call	0	0	Tie
Languages supported	1	4	**JSII**
Codebases to maintain	1 per lang	1 total	**JSII**

Table 7: **The full picture.** Green = JSII advantage. Purple = Native advantage. Red = significant Native advantage. JSII wins on speed, CPU efficiency, and polyglot; Native wins on total memory.

7 When Should You Use Which?

Choose Native When

- Memory is tight (<128 MB containers)
- Python-only deployment
- You need the smallest RSS
- You want zero Node.js dependency

Choose JSII When

- Multi-language SDK delivery
- Many short-lived agents (fast init)
- CPU-billed environments
- Single codebase for Py/Java/.NET/Go

Figure 7: **Decision guide.** Both are production-ready. The choice depends on your deployment constraints.

8 Caveats and Future Work

- **Single run.** Bedrock response times vary by ±2–3 s. A proper benchmark should average 10+ runs. We plan to automate this.
- **Node.js CPU not captured.** On macOS, `getrusage(RUSAGE_CHILDREN)` only reports CPU for *terminated* children. The JSII Node.js process is still running at measurement time, so its CPU shows as zero. A Linux `/proc`-based measurement would be more accurate.
- **Node 18 is EOL.** V8 memory overhead may decrease with Node 20+. We'll re-benchmark when the JSII runtime supports newer Node versions.
- **No warm-start benchmark.** We measured cold-start only. For long-running agents, the construction advantage diminishes and the memory overhead becomes more relevant.
- **VSZ is misleading.** V8's `-max-old-space-size=4069` causes each Node.js process to reserve ∼400 GB of *virtual* memory. This is not physical RAM — don't panic if you see it in `ps aux`.

Benchmark environment: Apple Silicon arm64, macOS Darwin, Python 3.13.12, Node.js 18.20.5, Claude Sonnet 4 via Amazon Bedrock (us-west-2, streaming). Prompt: "say hello 50 times" with single `hello(text)` tool triggering 50 tool call cycles. Memory measured via `resource.getrusage(RUSAGE_SELF)` for Python + `ps -o rss -p <pid>` for each child process. All source code and benchmark scripts available at `strands-jsii/benchmarks/`.